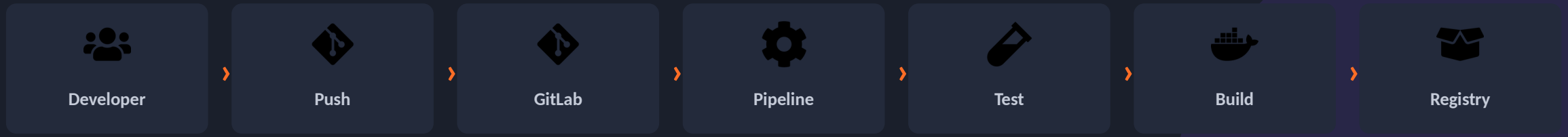
PHASE 4 · CI/CD SUPPLY CHAIN

Chapter 13

GitLab CE: Source Control & CI/CD Pipelines

Host: **sec-vcs-01 (10.0.2.14)** | Time estimate: **5–6 hours**



The Single Source of Truth

Code

All source code version-controlled

CI/CD

Automated testing and building

Security

SAST, DAST, container scanning

Collaboration

Code review before merge

Audit

Every commit, review, deploy logged

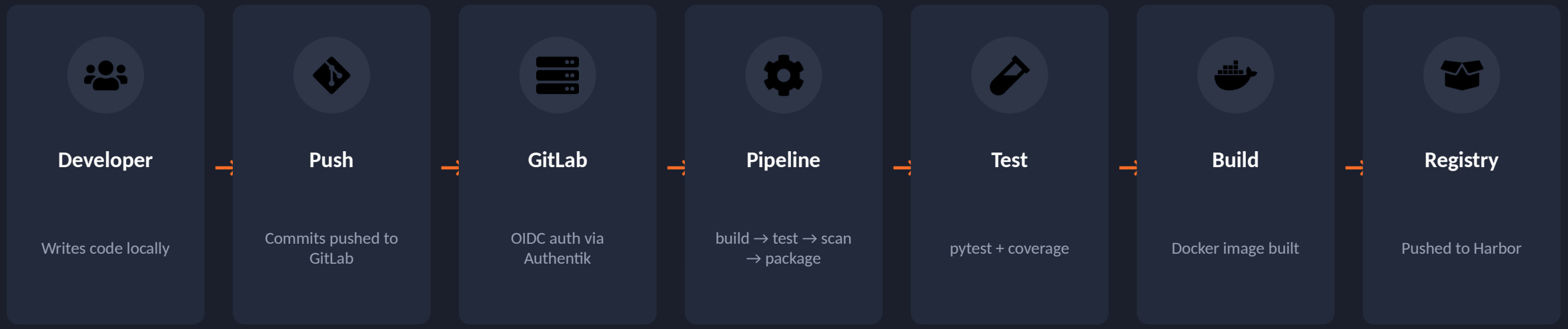
🏗️ **ANALOGY — GitLab is a Modern Factory**

- Source code = raw materials
- Merge requests = quality control
- CI/CD pipelines = assembly line
- Container registry = warehouse
- Deployments = shipping

13.1 · ARCHITECTURE OVERVIEW

Developer → Push → GitLab → CI/CD Pipeline → Test → Build → Registry

Chapter goal: GitLab operational as the central VCS + CI/CD engine on sec-vcs-01 (10.0.2.14)



By the end of this chapter: OIDC auth · versioned repos · automated pipelines · code review · Vault secrets · Docker builds · security scanning

Deploy GitLab CE with Docker Compose

1 Create GitLab Directory

```
mkdir -p /opt/gitlab/{config,logs,data}
```

2 Write docker-compose.yml

```
gitlab-ce, postgres:14, redis:7-alpine services on a shared 'devsecops' bridge network
```

3 Start the Stack

```
docker-compose up -d  →  watch logs 2-3 min for reconfigure
```

Config highlights: OIDC via Authentik issuer, external HAProxy for TLS (nginx disabled), registry disabled (Harbor used instead), 7-day backup retention.

First Login & Single Sign-On



Initial Setup

- Get initial root password from /etc/gitlab/initial_root_password
- Browse to https://gitlab.devsecops.com/ and log in as root
- Route traffic through HAProxy (backend be_gitlab → 10.0.2.14:80)
- Change the root password: Profile → Account → Password



OIDC via Authentik

- Authentik: Applications → Create Application "GitLab" (OAuth2/OIDC provider)
- Set Client ID / Secret to match docker-compose values
- Redirect URI: https://gitlab.devsecops.com/users/auth/oidc/callback
- Scopes: openid, profile, email
- gitlab-ctl reconfigure, then test login with LDAP users (john, alice, bob)

The Sample "myapp" Application

New Project → Blank Project · Name: myapp · Visibility: Private · git clone https://gitlab.devsecops.com/root/myapp.git



app.py

Flask app with /health and /api/message routes returning JSON



requirements.txt

Flask==2.3.0, gunicorn==20.1.0



Dockerfile

python:3.11-slim base → pip install → gunicorn entrypoint on :5000



tests/test_app.py

pytest fixtures asserting /health and /api/message responses

```
$ git add . && git commit -m "Initial application setup with CI/CD" && git push origin main
```

.gitlab-ci.yml — Four Pipeline Stages



Vault integration: before_script pulls DB_PASSWORD via curl + Vault token, parsed with jq — secrets never hardcoded in the pipeline.

Feeding Secrets Into the Pipeline

CI/CD Variables

Settings → CI/CD → Variables

- HARBOR_USERNAME
- HARBOR_PASSWORD
- SONARQUBE_TOKEN

Vault Integration

```
before_script:
  export DB_PASSWORD=$(curl -s \
    -H "X-Vault-Token: $VAULT_TOKEN" \
    https://vault.devsecops.com/v1/
    secret/data/database/mysql |
    jq -r '.data.data.password')
```

Secrets are fetched at pipeline runtime — never committed to the repo.

Docker-in-Docker

Enable shared runners for container builds via gitlab-rails runner:

```
shared_runners_enabled: true
runners_registration_token_
  reset_interval: 0
```

Code Review Before Merge

Branch

git checkout -b feature/new-endpoint

Commit

Add /api/newfeature route, commit changes

Push

git push origin feature/new-endpoint

Review

Open MR — reviewers see pipeline results

Merge

Merge only if all checks pass

GitLab enforces review before merge — reviewers see live pipeline status directly on the MR, so broken code never lands on main.

Next: enforce this with branch protection rules (see Security Best Practices).

Automating What Happens After Merge

Webhook Configuration

Project → Settings → Webhooks

- URL: https://deployment-system.devsecops.com/deploy
- Trigger: Merge request events, Push events
- SSL verification: enabled (valid cert required)

Each webhook call carries the event type, project, ref, and commit SHA as JSON.

Example Deployment Handler

```
@app.route('/deploy', methods=['POST'])
def deploy():
    data = request.json
    if data['event_type'] == 'push':
        deploy_image(
          data['project'],
          data['ref'],
          data['commit_sha'])
    return {'status': 'ok'}, 202
```

Protecting Against Data Loss

Create Backup

```
docker exec gitlab gitlab-backup create
SKIP=registry
```

Backups stored in /opt/gitlab/data/backups/

Retention: 7 days (604800s), configured in gitlab_rails['backup_keep_time'].

Restore From Backup

```
docker-compose down
docker-compose up -d postgres redis
sleep 10   # wait for DB init
gitlab-backup restore BACKUP=<file>
gitlab-ctl reconfigure
docker-compose restart gitlab
```

Hardening the Repository

Branch Protection

Require 1+ approvers, dismiss stale approvals, resolve all discussions, require pipeline success

Audit Logging

Limit Personal Access Token scopes to :api via gitlab-rails runner

HTTPS Only

TLS termination enforced at HAProxy edge

Signed Commits

Require signed commits: Settings → Repository

What We Built

What We Built	Purpose
GitLab CE server	Source code repository + CI/CD
OIDC integration	Authenticate with Authentik
CI/CD pipelines	Automated testing, scanning, building
Merge request workflow	Code review before merge
Docker integration	Build container images
Vault integration	Fetch secrets in pipelines
Webhooks	Trigger deployments
Audit logging	Track all changes



`validate-chapter-13.sh`

- GitLab container running
- Web UI accessible (curl)
- PostgreSQL container running
- Redis container running

🎉 *GitLab is operational!*
Access: https://gitlab.devsecops.com

KEY TAKEAWAY · CODE IS NOW SECURED

Before → After GitLab

BEFORE GITLAB

- No version control
- No code review
- No automated testing
- Manual deployments (error-prone)

AFTER GITLAB

- All code versioned
- Code review enforced
- Automated testing on every MR
- Automated building and deployment

NEXT CHAPTERS

Chapter 14 → SonarQube

Static code analysis: bugs, vulnerabilities, code smells before deployment.

Chapter 15 → Harbor

Container registry: stores Docker images and scans them for vulnerabilities.